VOLUME 06 OF 10

Game Systems and Production

Upper undergraduate / engineering capstone

Add simulation, animation, audio, AI, multiplayer, accessibility and release discipline.

5 modules / 20 lessons / 60 quiz questions

Prepared September 28, 2026

A guided self-study program with original lesson primers, assigned study, engineering labs, checkpoint projects and research work. Not an accredited degree or a substitute for the required practice and feedback.

Volume map

Module 26 Collision detection and physical simulation

26.1 Broad phase, narrow phase and geometric robustness • 26.2 Rigid bodies, angular motion and collision impulses • 26.3 Constraints, iterative solvers and continuous collision detection • 26.4 Physics testing, determinism and engine integration

Module 27 Animation, audio and interactive timing

27.1 Skeletons, transforms and skinning • 27.2 Animation blending, state machines and inverse kinematics • 27.3 Digital audio, mixing and real-time callback discipline • 27.4 Spatial audio, animation events and multimodal accessibility

Module 28 Game AI, decision systems and machine learning

28.1 Navigation graphs, steering and dynamic obstacles • 28.2 Behavior trees, utility systems and explainable decisions • 28.3 Adversarial search, planning and simulation budgets • 28.4 Learning systems, neural inference and responsible evaluation

Module 29 Multiplayer simulation and online services

29.1 Authority, prediction and reconciliation • 29.2 Snapshots, interpolation, relevance and lag compensation • 29.3 Lockstep, rollback and deterministic replay • 29.4 Match services, identity, persistence and failure recovery

Module 30 Production quality and the undergraduate engine capstone

30.1 Profiling, frame budgets and performance regression tests • 30.2 Compatibility, accessibility, localization and human testing • 30.3 Packaging, patching, modding and supply-chain boundaries • 30.4 Capstone review, release and technical postmortem

Before beginning

Use PROGRAM_GUIDE for sequencing, workload, assessment and the checkpoint project rubric. Use START_HERE for the first session. Reference IDs such as R07 link to assigned resources. The complete reference directory is in PROGRAM_GUIDE and the browser edition.

Answer guidance is separate in ANSWER_KEY.pdf. In the browser edition, answers remain closed until you reveal them. Finish the required study and implementation work rather than treating quiz recall as mastery.

MODULE 26

Collision detection and physical simulation

Prerequisites Modules 07-10, 14 and 24.

Four lessons

26.1 Broad phase, narrow phase and geometric robustness

Separate possible collisions from confirmed contacts.

26.2 Rigid bodies, angular motion and collision impulses

Use conservation laws with explicit approximations.

26.3 Constraints, iterative solvers and continuous collision detection

Explain where discrete simulation fails and how solvers control error.

26.4 Physics testing, determinism and engine integration

Validate simulation with invariants and adversarial scenes.

Module project

Build a deterministic test harness for a small physics scene, compare your simple implementation with a maintained physics library, and record precisely which physical and numerical assumptions differ.

Core references

R28 Box2D project — Box2D documentation

R19 Carnegie Mellon University — 15-462/662 Computer Graphics

R23 Matt Pharr, Wenzel Jakob, Greg Humphreys — Physically Based Rendering, fourth edition

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 26.1 • MODULE 26

Broad phase, narrow phase and geometric robustness

Learning goal Separate possible collisions from confirmed contacts.

A collision system first identifies potentially interacting pairs, then performs more expensive geometric tests. The broad phase must be conservative: extra candidate pairs cost time, but missing a real collision breaks correctness. Spatial grids, sweep-and-prune and bounding-volume hierarchies suit different distributions and movement patterns.

The narrow phase determines actual overlap or separation and, when needed, contact positions and normals. Contact normal orientation is an interface contract. Touching, overlapping and nearly parallel configurations need deliberate tolerance policies. Floating-point tolerances should relate to scale and purpose, not an unexplained universal constant. Keep a brute-force pair generator as an oracle for small tests before optimizing.

WORKED EXAMPLE

For 100 objects, checking every unordered pair requires 100 x 99 / 2 = 4,950 pair tests. A broad phase helps only if its maintenance cost and candidate count are favorable for the scene.

PRACTICE + PASS CRITERION

Implement a grid broad phase and circle or box narrow phase. Pass: no true pairs are omitted compared with brute force over randomized scenes, and boundary-touching behavior is documented.

ASSIGNED STUDY

R28: collision module, shapes and broad-phase concepts; R19: geometry and collision.

CHECK YOUR UNDERSTANDING

1. May a correct broad phase return false positives?

2. How many unordered pairs exist for 100 objects?

3. Why document contact-normal orientation?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 26.2 • MODULE 26

Rigid bodies, angular motion and collision impulses

Learning goal Use conservation laws with explicit approximations.

A rigid-body model represents position, orientation, linear velocity and angular velocity without modeling deformation. Mass relates force to linear acceleration, while an inertia tensor relates torque and angular acceleration under the chosen frame conventions. Off-center forces and impulses can change rotation as well as translation.

Collision impulses change velocity over an idealized short event. For a frictionless one-dimensional collision, restitution controls the ratio of separating to approaching relative speed under the model. Friction constrains tangential impulses and requires care around resting contact. Infinite-mass or static bodies are commonly represented through zero inverse mass rather than numerical infinity. Conservation tests are powerful when applied to scenarios that actually satisfy the assumptions.

WORKED EXAMPLE

A 2 kg body receiving a 6 N*s impulse through its center changes velocity by 3 m/s. The same impulse away from its center also changes angular momentum.

PRACTICE + PASS CRITERION

Implement center-of-mass impulses and a simple rotational case. Pass: momentum is conserved in isolated tests within tolerance, static bodies remain fixed, and energy behavior matches the chosen restitution model.

ASSIGNED STUDY

R28: rigid bodies, contacts and simulation; R19: physical simulation.

CHECK YOUR UNDERSTANDING

1. What velocity change follows from 6 N*s applied to 2 kg?

2. Why can an off-center impulse cause rotation?

3. Why use zero inverse mass for a static body?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 26.3 • MODULE 26

Constraints, iterative solvers and continuous collision detection

Learning goal Explain where discrete simulation fails and how solvers control error.

A constraint restricts allowed motion, such as maintaining a joint distance or preventing interpenetration. Iterative solvers repeatedly reduce constraint error using local updates. The number of iterations, ordering, timestep and warm starting influence convergence and apparent stiffness. A stable-looking scene is not proof of exact constraint satisfaction.

Discrete collision detection samples configurations at steps and can miss an object passing completely through a thin obstacle. Continuous methods examine motion over an interval or estimate a time of impact. They require motion assumptions and add cost. Use them selectively where speed and geometry justify them. Position correction should not silently create unbounded energy or conceal a broken velocity solver.

WORKED EXAMPLE

An object moving 30 m/s travels 0.5 m during a 1/60 s step. A wall much thinner than that can be crossed between sampled positions without either endpoint overlapping.

PRACTICE + PASS CRITERION

Create a fast-projectile tunneling test and a short chain of constraints. Pass: a continuous or swept test catches the projectile, and a plot or table shows constraint error versus iteration count and timestep.

ASSIGNED STUDY

R28: joints, solver settings and continuous collision; R19: numerical simulation.

CHECK YOUR UNDERSTANDING

1. How far does 30 m/s travel in 1/60 second?

2. Why can increasing solver iterations help?

3. Does CCD remove every numerical simulation problem?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 26.4 • MODULE 26

Physics testing, determinism and engine integration

Learning goal Validate simulation with invariants and adversarial scenes.

Physics tests should include resting stacks, extreme but supported mass ratios, repeated contacts, long runtimes and objects entering or leaving the world. Define units and supported scales before tuning. NaNs or exploding positions should produce a diagnostic near their origin, not only a later renderer crash.

Deterministic replay requires control of ordering, arithmetic assumptions, random sources and inputs. Bitwise determinism across platforms is a stronger requirement than visually similar behavior on one machine. A library can have explicit determinism limits; record them. Compare analytic solutions where available and use convergence studies elsewhere. Integration tests must verify that rendering interpolation, collision events and entity destruction do not violate the physics system's lifetime rules.

WORKED EXAMPLE

A freely moving body with no force should maintain velocity in the ideal model. A falling body before collision can be checked against an analytic constant-acceleration solution, separating integration error from collision behavior.

PRACTICE + PASS CRITERION

Build a regression suite with replay hashes on one pinned environment, finite-value checks and analytic tests. Pass: failures identify the first divergent tick and reports distinguish same-machine repeatability from cross-platform determinism.

ASSIGNED STUDY

R28: simulation, determinism and limitations; revisit Modules 08 and 24.

CHECK YOUR UNDERSTANDING

1. Is same-machine repeatability proof of cross-platform determinism?

2. Why identify the first divergent tick?

3. What should happen when a simulation first creates a NaN?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 27

Animation, audio and interactive timing

Prerequisites Modules 07, 10, 18 and 24.

Four lessons

27.1 Skeletons, transforms and skinning

Compute hierarchical animation without mixing coordinate spaces.

27.2 Animation blending, state machines and inverse kinematics

Coordinate authored motion with runtime goals.

27.3 Digital audio, mixing and real-time callback discipline

Generate sound without missing the output deadline.

27.4 Spatial audio, animation events and multimodal accessibility

Synchronize feedback while preserving user control.

Module project

Integrate an animated character or articulated object with event-driven audio. Demonstrate smooth interpolation, bounded audio work, adjustable sound categories and correct behavior when animation rate differs from simulation rate.

Core references

R19 Carnegie Mellon University — 15-462/662 Computer Graphics

R20 Carnegie Mellon University — 15-466/666 Computer Game Programming

R22 SDL project — SDL3 documentation

R26 Epic Games — Programming with C++ in Unreal Engine

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 27.1 • MODULE 27

Skeletons, transforms and skinning

Learning goal Compute hierarchical animation without mixing coordinate spaces.

A skeleton is a hierarchy of local transforms. A joint's world transform depends on its ancestors, so evaluation must respect the hierarchy. Skinning transforms mesh vertices using joint transforms and stored bind information. Inverse bind matrices relate the original mesh pose to the current skeleton; applying a transform in the wrong space creates characteristic stretching or displacement.

Linear blend skinning combines transformed positions using weights that normally sum to one. It is efficient but has deformation limitations. Rotations should be interpolated with a representation appropriate to orientation, such as normalized quaternions with a shortest-path convention. Interpolating arbitrary matrix entries can introduce unintended scale or shear. Test a two-joint example before importing a complex character.

WORKED EXAMPLE

A vertex influenced equally by two joints has weights 0.5 and 0.5. Omitting the bind-pose correction can move it even when the skeleton is supposedly in its original pose.

PRACTICE + PASS CRITERION

Animate a two-joint mesh or line skeleton. Pass: the bind pose reproduces the source geometry, weights are validated and a parent rotation moves descendants consistently.

ASSIGNED STUDY

R19: transformations and animation; R26: skeletal animation concepts.

CHECK YOUR UNDERSTANDING

1. Why evaluate parent transforms before dependent children?

2. What should ordinary normalized skinning weights sum to?

3. Why avoid directly interpolating arbitrary rotation-matrix entries?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 27.2 • MODULE 27

Animation blending, state machines and inverse kinematics

Learning goal Coordinate authored motion with runtime goals.

Animation state machines choose motion states and transitions, while blend systems combine clips using parameters such as speed or direction. Transition conditions, interruption rules and event timing are part of gameplay behavior. A clip completing is not always equivalent to an action becoming interruptible.

Inverse kinematics finds joint configurations that satisfy target constraints, such as placing a foot on a surface. It may have no solution, multiple solutions or a solution outside joint limits. Iterative methods need stopping criteria and stable behavior near unreachable targets. Root motion introduces a policy decision about whether animation or simulation controls displacement. Keep ownership clear to prevent animation and physics from both moving the same body inconsistently.

WORKED EXAMPLE

A two-link arm with lengths 1 and 1 cannot reach a target 3 units from its base. A solver should clamp or report the unreachable condition rather than generating invalid angles.

PRACTICE + PASS CRITERION

Build idle/walk/action transitions and a two-link IK demonstration. Pass: interruptions behave as specified, unreachable targets are handled, and root-motion ownership is documented.

ASSIGNED STUDY

R19: animation and inverse kinematics; R26: animation blueprints and blending concepts, compared with your own small implementation.

CHECK YOUR UNDERSTANDING

1. Can every IK target be reached?

2. What is the maximum reach of two collinear links of length 1 each?

3. Why define root-motion ownership?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 27.3 • MODULE 27

Digital audio, mixing and real-time callback discipline

Learning goal Generate sound without missing the output deadline.

Digital audio represents sampled signals. Sample rate, channel layout and sample format describe the buffer contract. Mixing combines signals with controlled gain; clipping occurs when output exceeds the representable or intended range. Resampling is a signal-processing operation requiring filtering, not merely dropping arbitrary samples.

An audio callback usually has a deadline tied to the buffer duration. Avoid unbounded allocation, blocking locks, disk access or unpredictable work on that path. Prepare data elsewhere and communicate through a carefully designed bounded mechanism. Smaller buffers can reduce one source of latency but leave less scheduling margin. Measure underruns and end-to-end response rather than assuming the smallest available buffer is best.

WORKED EXAMPLE

A 256-frame buffer at 48,000 frames per second spans 256/48,000 seconds, about 5.33 ms. A callback that blocks for 20 ms can miss that timing requirement.

PRACTICE + PASS CRITERION

Mix two generated tones with adjustable gains using a platform audio API. Pass: no clipping under the documented gain policy, bounded callback work and a recorded underrun or deadline-stress test.

ASSIGNED STUDY

R10 from Volume 2: sampling and filtering; R22: SDL3 audio; R20: audio systems.

CHECK YOUR UNDERSTANDING

1. What duration does 256 frames represent at 48 kHz?

2. Why avoid disk reads inside an audio callback?

3. Does halving the buffer automatically improve every aspect of audio?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 27.4 • MODULE 27

Spatial audio, animation events and multimodal accessibility

Learning goal Synchronize feedback while preserving user control.

Spatial audio approximates cues such as level, direction and environmental filtering. A simple stereo pan is not a full head-related acoustic model. Distance attenuation and occlusion are design choices that should be consistent with the game's spatial scale and readability needs. Avoid treating physically plausible sound as automatically useful feedback.

Animation and gameplay events should reference a deliberate clock and deduplication policy. Rollback or repeated evaluation must not replay every sound unintentionally. Provide category volume controls and alternatives for information communicated only through sound or color. Subtitle timing, speaker identification and visual event indicators are engineering requirements, not a final decoration. Evaluate accessibility with actual task completion where possible.

WORKED EXAMPLE

A predicted footstep may be encountered again during replay. Assigning an event identity and a presentation policy prevents an unintended duplicate sound while preserving the corrected simulation.

PRACTICE + PASS CRITERION

Add spatial panning, separate music/effects controls and a visual equivalent for a critical audio cue. Pass: replay does not duplicate one-shot events and a muted-audio test still allows the required task.

ASSIGNED STUDY

R20: audio and event systems; R22: audio interfaces; R29 from the resource directory: audio and subtitle accessibility.

CHECK YOUR UNDERSTANDING

1. Is stereo panning a complete spatial-hearing model?

2. Why give presentation events stable identities during replay?

3. Why provide a non-audio equivalent for a critical cue?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 28

Game AI, decision systems and machine learning

Prerequisites Modules 09, 14, 24 and 26.

Four lessons

28.1 Navigation graphs, steering and dynamic obstacles

Separate finding a route from executing movement.

28.2 Behavior trees, utility systems and explainable decisions

Represent behavior so designers can understand its failures.

28.3 Adversarial search, planning and simulation budgets

Trade decision quality against bounded computation.

28.4 Learning systems, neural inference and responsible evaluation

Understand learning fundamentals without replacing every subsystem with a model.

Module project

Build an inspectable agent that navigates, chooses actions and recovers from blocked plans. Compare a rules-based baseline with one small learned or search-based component; evaluate behavior and runtime cost, not only win rate.

Core references

R20 Carnegie Mellon University — 15-466/666 Computer Game Programming

R46 University of California, Berkeley — CS188 Introduction to Artificial Intelligence textbook

R47 Aston Zhang, Zachary Lipton, Mu Li and Alexander Smola — Dive into Deep Learning

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 28.1 • MODULE 28

Navigation graphs, steering and dynamic obstacles

Learning goal Separate finding a route from executing movement.

Pathfinding chooses a route through an abstract representation; steering controls local motion along or toward that route. A navigation mesh represents traversable regions, while grids and waypoint graphs offer other tradeoffs. The abstraction must include the agent's dimensions and movement constraints. A path valid for a point may be impossible for a wide character.

Dynamic obstacles can invalidate a route or require local avoidance. Replanning every frame for every agent may waste time; never replanning may leave agents stuck. Use bounded work, event-driven invalidation and observable failure states. Local avoidance does not guarantee a globally feasible route. Test narrow passages, opposing traffic and unreachable destinations rather than only empty rooms.

WORKED EXAMPLE

A doorway 0.8 units wide is not traversable by a circular agent with diameter 1.0 under a no-overlap model, even if the navigation graph contains an edge through its center.

PRACTICE + PASS CRITERION

Combine A* with a steering controller and a blocked-path detector. Pass: agent size is respected, unreachable goals terminate or report failure, and repeated replanning has a bounded budget.

ASSIGNED STUDY

R46: Search; R20: game AI and navigation concepts.

CHECK YOUR UNDERSTANDING

1. How do pathfinding and steering differ?

2. Can the example 1.0-wide agent pass through the 0.8-wide doorway?

3. Why bound replanning work?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 28.2 • MODULE 28

Behavior trees, utility systems and explainable decisions

Learning goal Represent behavior so designers can understand its failures.

A finite-state machine encodes states and transitions. A behavior tree organizes conditional execution and action status. A utility system scores alternatives and selects according to a policy. These are alternative or complementary organizational tools, not a ranking from obsolete to modern. Choose based on behavior complexity, authoring needs and debugging.

An action should expose progress, success, failure and cancellation semantics. Utility scores need compatible scales and often hysteresis to prevent rapid switching between nearly equal options. Behavior traces should record why an action was selected and why it failed. A black-box agent that occasionally behaves well is harder to improve than a simpler agent whose decisions can be inspected.

WORKED EXAMPLE

An agent choosing between attack score 0.51 and retreat score 0.50 can oscillate as small observations change. A switching threshold or minimum commitment interval can stabilize the policy, with explicit tradeoffs.

PRACTICE + PASS CRITERION

Implement either a small behavior tree or utility selector with a trace viewer. Pass: action cancellation releases resources and three induced failure cases produce understandable explanations.

ASSIGNED STUDY

R20: game AI; R46: agents, utilities and decision-making foundations.

CHECK YOUR UNDERSTANDING

1. What statuses should a multi-frame action expose?

2. Why can nearly tied utility scores cause oscillation?

3. Why is a decision trace valuable?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 28.3 • MODULE 28

Adversarial search, planning and simulation budgets

Learning goal Trade decision quality against bounded computation.

Minimax reasons about alternating choices under a specified game model. Alpha-beta pruning can avoid exploring branches that cannot affect the result while preserving the minimax value under the same assumptions. Move ordering changes efficiency, not the objective. Evaluation functions approximate the value of states that cannot be searched to completion.

Monte Carlo tree search uses repeated sampled exploration to allocate work. Its behavior depends on sampling, exploration policy and rollout or value estimates. Real-time agents need deadlines and a valid fallback action. Hidden information requires reasoning about beliefs rather than granting the agent access to facts a player could not observe. Compare policies on diverse scenarios, not only against one weak opponent.

WORKED EXAMPLE

A branching factor of 10 searched to depth 6 can produce roughly one million leaf paths before pruning. A 2 ms decision budget cannot be assumed sufficient without measurement and a bounded policy.

PRACTICE + PASS CRITERION

Implement minimax with alpha-beta or a small MCTS agent for a tiny game. Pass: legal moves are always returned within a defined budget or fallback, and evaluation includes multiple seeds and opponents.

ASSIGNED STUDY

R46: Games, Minimax, Expectimax and Monte Carlo Tree Search.

CHECK YOUR UNDERSTANDING

1. Does alpha-beta change the minimax objective when applied correctly?

2. How many depth-6 paths arise from branching factor 10 without pruning?

3. Why use a fallback action?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 28.4 • MODULE 28

Learning systems, neural inference and responsible evaluation

Learning goal Understand learning fundamentals without replacing every subsystem with a model.

Supervised learning fits parameters from examples; reinforcement learning optimizes behavior through rewards and interaction. A training objective is not the same as the desired player experience. Split data by independent episodes or sources when appropriate so nearly duplicate examples do not leak between training and testing. Establish a simple non-learned baseline first.

A neural network composes parameterized transformations and nonlinearities; gradients update parameters through the chain rule. Inference cost includes preprocessing, memory movement and scheduling, not only arithmetic. Distribution shift can cause failure when gameplay differs from training. Record data rights, privacy and failure policies. Learned components should remain bounded and observable, especially when their output affects authoritative game state.

WORKED EXAMPLE

A model trained and tested on adjacent frames from the same recorded match may appear accurate by exploiting near-duplicates. Holding out complete matches provides a stronger test of generalization.

PRACTICE + PASS CRITERION

Train a small classifier or value estimator on synthetic game data and compare it with a rules baseline. Pass: independent splits, error analysis, inference timing, reproducible seeds and a fallback behavior are reported.

ASSIGNED STUDY

R46: Machine Learning and Reinforcement Learning; R47: linear models, multilayer perceptrons, backpropagation and generalization.

CHECK YOUR UNDERSTANDING

1. Why separate complete episodes across data splits?

2. What mathematical rule underlies backpropagation?

3. Why retain a rules-based baseline?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 29

Multiplayer simulation and online services

Prerequisites Modules 18-20 and 24; Module 26 for physical interaction.

Four lessons

29.1 Authority, prediction and reconciliation

Provide responsive local control without trusting arbitrary client outcomes.

29.2 Snapshots, interpolation, relevance and lag compensation

Manage bandwidth and latency while making fairness assumptions explicit.

29.3 Lockstep, rollback and deterministic replay

Choose synchronization around the game model rather than genre slogans.

29.4 Match services, identity, persistence and failure recovery

Design a small online service without treating it as part of the render loop.

Module project

Create a local two-client multiplayer demonstration with a network simulator for delay, jitter, loss, duplication and reordering. No public server or paid cloud account is required. Document authority and recovery rules.

Core references

R17 IETF / RFC Editor — RFC 9293: Transmission Control Protocol

R20 Carnegie Mellon University — 15-466/666 Computer Game Programming

R26 Epic Games — Programming with C++ in Unreal Engine

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 29.1 • MODULE 29

Authority, prediction and reconciliation

Learning goal Provide responsive local control without trusting arbitrary client outcomes.

An authoritative simulation decides which state is accepted as game truth. Clients commonly send input intentions rather than unrestricted final positions or rewards. Local prediction improves immediate response by simulating the client's own input before server confirmation. Reconciliation compares authoritative state with the corresponding predicted history and reapplies unacknowledged inputs when necessary.

Input sequence numbers or simulation ticks make this correspondence possible. Validation needs resource and rate limits as well as semantic checks. Authority does not eliminate all cheating or fairness problems, but it defines the trust boundary. Presentation smoothing should not rewrite the authoritative rules. Keep correction behavior visible during development so errors are not merely hidden behind interpolation.

WORKED EXAMPLE

The server acknowledges input 40. The client corrects to the returned state for that point and replays locally stored inputs 41 through 44, rather than applying those inputs twice to the wrong starting state.

PRACTICE + PASS CRITERION

Implement a one-dimensional or grid movement server with client prediction. Pass: invalid movement is rejected and induced delay produces documented corrections without permanently diverging state.

ASSIGNED STUDY

R17: transport semantics; R26: networking and multiplayer concepts; R20: networking in games.

CHECK YOUR UNDERSTANDING

1. What is authoritative state?

2. Which inputs are replayed after acknowledgment of input 40 in the example?

3. Why send intentions rather than unrestricted reward totals?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 29.2 • MODULE 29

Snapshots, interpolation, relevance and lag compensation

Learning goal Manage bandwidth and latency while making fairness assumptions explicit.

Remote entities can be rendered between buffered snapshots, trading some added presentation delay for smoother motion under jitter. Extrapolation predicts beyond available data and can be wrong when motion changes. Snapshot sequence numbers, timestamps and baselines must be validated. Delta compression requires agreement about the baseline being referenced.

Interest management sends relevant state rather than every entity to every client. Relevance is also a security concern: sending hidden information can expose it even when the interface does not display it. Lag compensation may evaluate an action against historical state, but history bounds and fairness policy must be explicit. There is no universal policy that makes every latency condition equally fair.

WORKED EXAMPLE

Buffering two snapshots 50 ms apart can interpolate motion during a delay variation, but it intentionally displays a slightly older world. This is a latency-quality tradeoff, not free smoothing.

PRACTICE + PASS CRITERION

Add remote interpolation and relevance filtering. Pass: out-of-order snapshots are handled, missing baselines trigger recovery, and the report explains both visual delay and hidden-state exposure policy.

ASSIGNED STUDY

R20: networked game state; R26: replication and network smoothing; revisit Module 19 message validation.

CHECK YOUR UNDERSTANDING

1. What cost does snapshot interpolation commonly add?

2. Why must delta-compression baselines be identified?

3. Why avoid transmitting irrelevant hidden enemy state?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 29.3 • MODULE 29

Lockstep, rollback and deterministic replay

Learning goal Choose synchronization around the game model rather than genre slogans.

Lockstep peers or participants advance a deterministic simulation using agreed inputs. It can reduce state bandwidth, but progress depends on input availability and determinism. A single divergent arithmetic or ordering decision can amplify into different worlds. Periodic state hashes help detect disagreement but do not repair it by themselves.

Rollback predicts missing inputs, then restores a past state and resimulates when the prediction differs. It needs fast state capture, bounded history and careful handling of irreversible presentation or external side effects. The rollback horizon controls memory and correction cost. Neither rollback nor lockstep is automatically appropriate for every RTS or action game; evaluate state size, simulation cost, latency and player experience.

WORKED EXAMPLE

At 60 simulation ticks per second, storing 120 snapshots covers two seconds of history. If each snapshot is 2 MB, raw snapshot storage alone reaches 240 MB before metadata or compression.

PRACTICE + PASS CRITERION

Implement rollback or input lockstep for the tiny deterministic game. Pass: a delayed-input test converges, state hashes detect a deliberately introduced divergence, and audio or reward side effects are not duplicated.

ASSIGNED STUDY

R20 and R26: multiplayer concepts; derive the implementation using Modules 18, 24 and 26 rather than assuming engine-specific guarantees.

CHECK YOUR UNDERSTANDING

1. How much time do 120 ticks cover at 60 Hz?

2. How much raw storage do 120 snapshots of 2 MB require?

3. Why are external side effects difficult during rollback?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 29.4 • MODULE 29

Match services, identity, persistence and failure recovery

Learning goal Design a small online service without treating it as part of the render loop.

Matchmaking, authentication, session allocation and persistent progression are service responsibilities distinct from frame simulation. A match can continue or fail under a defined policy when a nonessential service is unavailable. Identity and authorization must be checked at the relevant boundary; a valid account is not permission to modify another account's data.

Retries, timeouts and idempotency are essential for reward and progression operations. Use established authentication and cryptographic libraries rather than inventing protocols. Store secrets outside repositories and logs. Operational readiness includes health checks, structured diagnostics, graceful shutdown and recovery tests. For learning, local mock services and synthetic accounts are sufficient and avoid unnecessary privacy and spending risks.

WORKED EXAMPLE

A match-result submission is retried after a timeout. A stable operation identifier lets the progression service recognize the duplicate and avoid granting the same reward twice.

PRACTICE + PASS CRITERION

Build a local match registry and idempotent result store. Pass: duplicate submissions do not duplicate rewards, unauthorized edits fail, and simulated service loss produces a documented client outcome.

ASSIGNED STUDY

R18 from the directory: transactions; R17: transport; revisit Modules 19-20 for distributed failure and threat modeling.

CHECK YOUR UNDERSTANDING

1. Is authentication the same as authorization?

2. Why identify a match-result operation uniquely?

3. Must this lab use a paid public cloud?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 30

Production quality and the undergraduate engine capstone

Prerequisites Modules 21-29 plus Gate D software-engineering evidence.

Four lessons

30.1 Profiling, frame budgets and performance regression tests

Optimize measured bottlenecks without sacrificing correctness.

30.2 Compatibility, accessibility, localization and human testing

Test whether people can actually use the complete product.

30.3 Packaging, patching, modding and supply-chain boundaries

Distribute a game without creating an uncontrolled execution path.

30.4 Capstone review, release and technical postmortem

Demonstrate ownership of a complete engineering result.

Module project

Gate F: release a small original game using your engine, with one complete gameplay loop, settings, save/load, diagnostics, accessibility checks, a tested distribution and a technical postmortem. Scope is judged by quality of evidence, not visual size.

Core references

R20 Carnegie Mellon University — 15-466/666 Computer Game Programming

R26 Epic Games — Programming with C++ in Unreal Engine

R29 Microsoft — Xbox Accessibility Guidelines

R45 LLVM Project — AddressSanitizer documentation

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 30.1 • MODULE 30

Profiling, frame budgets and performance regression tests

Learning goal Optimize measured bottlenecks without sacrificing correctness.

Profiling identifies where time and resources are spent under a specific workload. Instrumentation records explicit events; sampling estimates where execution tends to be observed. Both have overhead and interpretation limits. A CPU trace alone cannot fully explain GPU work, and a GPU timing alone cannot describe input-to-display latency.

Create repeatable scenes with fixed settings and a warmup policy. Record build type, hardware, drivers, resolution and background conditions. Measure distributions and peak memory, then change one hypothesis-driven factor at a time. A microbenchmark speedup is useful only if it improves an important application path without unacceptable regressions. Keep correctness tests alongside performance tests and tolerate normal measurement noise appropriately.

WORKED EXAMPLE

A subsystem taking 2 ms of a 20 ms frame can save at most 2 ms if removed entirely. A 10x speedup in that subsystem alone yields an 18.2 ms frame under the simplified additive model.

PRACTICE + PASS CRITERION

Profile the capstone scene, optimize one verified bottleneck and add a regression benchmark. Pass: repeated before/after results, unchanged correctness and disclosed measurement uncertainty are included.

ASSIGNED STUDY

R26: profiling and optimization; R45: sanitizer diagnostics; revisit Modules 09 and 18.

CHECK YOUR UNDERSTANDING

1. What is the ideal frame time after a 10x speedup of the example 2 ms subsystem?

2. Why record build type?

3. Why can a microbenchmark improvement fail to help the game?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 30.2 • MODULE 30

Compatibility, accessibility, localization and human testing

Learning goal Test whether people can actually use the complete product.

A compatibility matrix lists supported environments and input/display configurations. Test cases should include different resolutions, aspect ratios, controllers, keyboard layouts and missing optional devices where relevant. Unsupported cases need clear behavior rather than accidental crashes. Localization affects text length, fonts, layout, number formatting and sometimes text direction.

Accessibility begins with requirements such as remappable controls, readable interface scale, contrast, alternative cues and adjustable timing where the game permits. A guideline checklist supports design but does not replace user testing or establish legal compliance. Use informed, voluntary test participation and collect only necessary data. Observe task completion and confusion rather than asking only whether the game is enjoyable.

WORKED EXAMPLE

A button sized for a short English label can clip a longer translated label. A control prompt that hard-codes a keyboard letter can become wrong after rebinding.

PRACTICE + PASS CRITERION

Test the game at two window sizes, with remapped input and an expanded pseudo-localized text set. Pass: no critical clipped controls, a muted-audio task remains solvable and observed usability issues are tracked.

ASSIGNED STUDY

R29: input, text, audio, contrast and UI guidelines; R20: playtesting and production.

CHECK YOUR UNDERSTANDING

1. Why use pseudo-localization before full translation?

2. What should a control prompt show after rebinding?

3. Does checking every accessibility guideline prove legal compliance?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 30.3 • MODULE 30

Packaging, patching, modding and supply-chain boundaries

Learning goal Distribute a game without creating an uncontrolled execution path.

A release package includes the executable, required runtime assets, dependency information, notices and configuration defaults. Test installation and startup outside the development tree. Patch systems need integrity checks, version compatibility and a rollback or recovery plan; interrupted updates must not destroy the only runnable installation.

Modding expands the trust boundary. Data-only mods still require parsing and resource limits, while executable plugins can run with substantial authority unless isolated. Define supported APIs, compatibility versions and permissions. Do not download and execute arbitrary community code automatically. A script sandbox needs explicit capability and resource limits, and its security depends on implementation, not the word sandbox. Preserve user saves separately from replaceable program files.

WORKED EXAMPLE

A malformed mod declares a texture with dimensions large enough to overflow a byte-count calculation. Validation must check multiplication and total resource limits before allocation, even though the file is nominally data.

PRACTICE + PASS CRITERION

Package the game and implement a data-only mod with a versioned schema. Pass: a clean-folder launch works, oversized or malformed mods are rejected and an interrupted-update simulation preserves a valid version.

ASSIGNED STUDY

R20: production; R26: packaging and plugins; R45 and Module 20: defensive memory and dependency practices.

CHECK YOUR UNDERSTANDING

1. Can a data-only mod still be dangerous to robustness?

2. Why separate saves from replaceable installation files?

3. Does naming a system a sandbox establish isolation?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 30.4 • MODULE 30

Capstone review, release and technical postmortem

Learning goal Demonstrate ownership of a complete engineering result.

A capstone combines requirements, implementation, tests, performance evidence and communication. Freeze a small scope: one complete loop with a clear beginning, success/failure and restart is enough. Review the architecture against actual use, not only the original diagram. List dependencies and distinguish original code from libraries or assistance.

A technical postmortem explains what worked, what failed, evidence behind decisions and what would change next. Avoid rewriting the project history as an uninterrupted success story. An oral review should include a live bug investigation, a design tradeoff and a small code modification. These reveal understanding beyond a polished video. Graduation to advanced work requires the practical artifact and reasoning, not only lesson attendance.

WORKED EXAMPLE

A small game with reproducible builds, bounded memory, clear controls and an honest bug list is stronger evidence of engineering ownership than an unfinished large world with no tests.

PRACTICE + PASS CRITERION

Complete Gate F and the oral questions in the handbook. Pass: another tester finishes a full loop, the release is reproducible, critical defects are resolved or clearly blocked from release, and the postmortem cites measured evidence.

ASSIGNED STUDY

R20: course projects and critique; revisit the project rubric and professional-practice lessons.

CHECK YOUR UNDERSTANDING

1. What proves completion beyond a recorded demo?

2. Why disclose library and AI assistance?

3. Why include a live modification in an oral review?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.